

Day 4: Bitwise Operations & Register Manipulation

Platform: nRF52840 | RTOS: Zephyr RTOS

Learning Objectives

Bitwise AND (&)

Used to **read or clear** specific bits without affecting others:

```
uint32_t status = 0xABCD1234UL;
uint32_t masked = status & 0x0000FF00UL; // Keep only bits [15:8]
reg &= ~BIT(5);                          // Clear bit 5 only
```

Rule: AND with 1 keeps the bit, AND with 0 clears it.

Bitwise OR (|)

Used to **set** specific bits without affecting others:

```
reg |= BIT(3);           // Set bit 3
reg |= (0x7UL << 4);     // Set bits [6:4]
```

Rule: OR with 1 sets the bit, OR with 0 leaves it unchanged.

Bitwise XOR (^)

Used to **toggle** bits:

```
reg ^= BIT(7);           // Toggle bit 7
```

Rule: XOR with 1 flips the bit, XOR with 0 leaves it unchanged.

Bitwise NOT (~)

Inverts all bits — used with AND to clear bits:

```
uint32_t mask = ~BIT(3);    // 0xFFFFFFFF7 — all ones except bit 3
reg &= mask;                // Clears bit 3
```

Left Shift (<<) and Right Shift (>>)

```
uint32_t bit3 = 1UL << 3;   // 0x00000008 — bit 3 set
uint8_t field = (reg >> 4) & 0x0FU; // Extract bits [7:4]
```

Always use `UL` suffix to avoid signed shift bugs on 32-bit values.

BIT(n) Macro (Zephyr sys/util.h)

Zephyr provides safe, portable bit macros:

```
#include <zephyr/sys/util.h>
BIT(5)           // (1UL << 5)
GENMASK(7, 4)    // bits [7:4] = 0x000000F0
FIELD_GET(val, mask, shift)
```

Always prefer `BIT(n)` over `(1 << n)` — it's 32-bit safe and self-documenting.

Real nRF52840 Register Manipulation

The nRF52840 GPIO output register (`NRF_P0->OUT`) is a 32-bit register where each bit controls one pin:

```
NRF_P0->OUT |= BIT(13); // Set P0.13 (LED0 on nRF52840DK)
NRF_P0->OUT &= ~BIT(13); // Clear P0.13
NRF_P0->OUT ^= BIT(13);  // Toggle P0.13
```

In Zephyr, you'd use `gpio_pin_set_dt()` instead, but understanding the underlying register is essential.

Practice Tasks

1. Write macros to set/clear/toggle/test any bit in any uint32_t register
 2. Extract bits [11:8] from the value `0x00000A5CUL` using shifts and masks
 3. Build a 32-bit SPI command packet: [cmd:8][addr:16][data:8] using bit operations
 4. Access `NRF_P0->OUT` directly (bypassing Zephyr) to toggle an LED — compare with GPIO API
-

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day04
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)
- Source: [src/day04/main.cpp](#)